

Aarya Arban

T11 – 05

Assignment No. 7

Aim: To register and install a service worker in the web app.

Theory:

Registering and Installing a Service Worker in a Web App

Service workers are a powerful feature that enable offline functionality, background synchronization, and push notifications in web applications. Here's how to register and install a service worker:

Step 1: Create a Service Worker File

First, create a JavaScript file for your service worker (typically named service-worker.js or sw.js) in your project's root directory:

```
// service-worker.js
const CACHE_NAME = 'my-app-v1';
const urlsToCache = [
  '/',
  '/index.html',
  '/styles/main.css',
  '/scripts/main.js',
  '/images/logo.png'
];
```

```
// Installation event - caching resources
self.addEventListener('install', event => {
  event.waitUntil(
    caches.open(CACHE_NAME)
      .then(cache => {
        console.log('Cache opened');
        return cache.addAll(urlsToCache);
      })
  );
});
```

```
// Activation event - clean up old caches
self.addEventListener('activate', event => {
  event.waitUntil(
    caches.keys().then(cacheNames => {
      return Promise.all(
        cacheNames.filter(name => {
          return name !== CACHE_NAME;
        }).map(name => {
          return caches.delete(name);
        })
      );
    })
  );
});
```

```

// Fetch event - serve from cache or network
self.addEventListener('fetch', event => {
  event.respondWith(
    caches.match(event.request)
      .then(response => {
        // Cache hit - return response
        if (response) {
          return response;
        }
        return fetch(event.request).then(
          response => {
            // Check if we received a valid response
            if (!response || response.status !== 200 || response.type !== 'basic') {
              return response;
            }

            // Clone the response
            const responseToCache = response.clone();

            caches.open(CACHE_NAME)
              .then(cache => {
                cache.put(event.request, responseToCache);
              });

            return response;
          }
        );
      })
  );
});

```

Step 2: Register the Service Worker

Add the following JavaScript code to your main application script (e.g., main.js or inline in your HTML):

```
if ('serviceWorker' in navigator) {  
  window.addEventListener('load', () => {  
    navigator.serviceWorker.register('/service-worker.js')  
      .then(registration => {  
        console.log('Service Worker registered with scope:', registration.scope);  
      })  
      .catch(error => {  
        console.error('Service Worker registration failed:', error);  
      });  
  });  
}
```

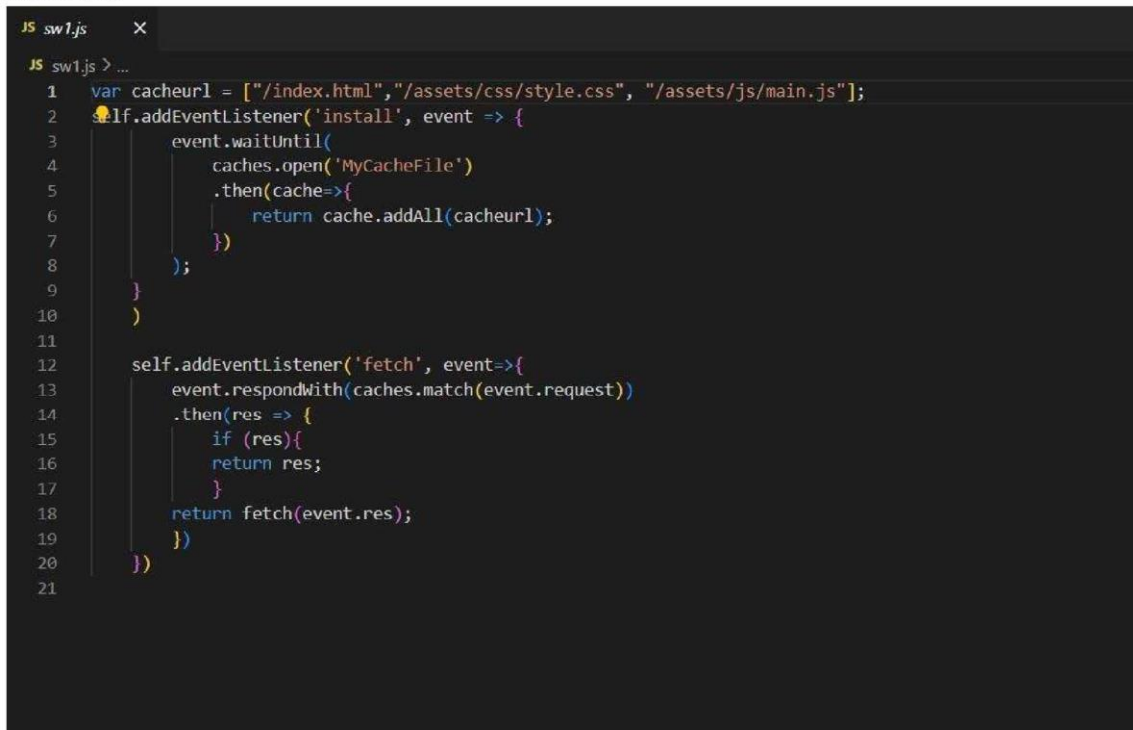
Step 3: Test Your Service Worker

1. Your website must be served over HTTPS (except for localhost, which works for development)
2. Open your browser's Developer Tools
3. Navigate to the Application tab (in Chrome) or similar in other browsers
4. Check the Service Workers section to verify registration

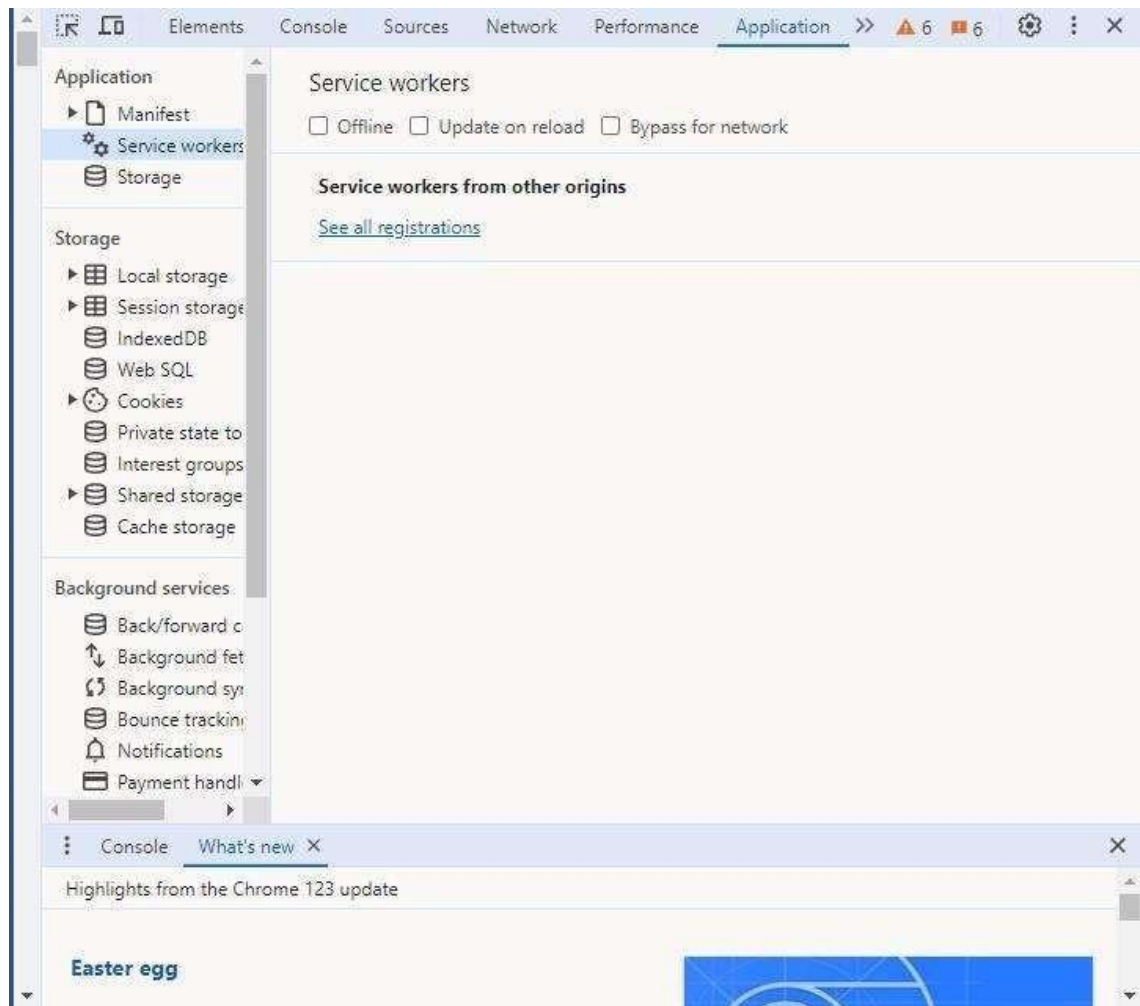
- **Offline Access:** By serving cached resources when offline, web applications remain functional even in the absence of an internet connection, enhancing accessibility and usability.
- **Reduced Bandwidth Usage:** Caching resources locally reduces bandwidth usage and server load, benefiting both users and web application developers.
- **Enhanced Reliability:** With the ability to store resources locally, web applications are more resilient to network failures and server downtime, ensuring continuous availability for users.

In conclusion, the Cache API, when used in conjunction with Service Workers, empowers web developers to create robust, reliable, and performant web applications that offer offline access and an enhanced user experience. By leveraging caching strategies, developers can optimize resource delivery, reduce dependency on network connectivity, and provide users with seamless access to content, even in challenging network conditions.

OUTPUT:



```
JS sw1.js
JS sw1.js > ...
1  var cacheurl = ["/index.html", "/assets/css/style.css", "/assets/js/main.js"];
2  self.addEventListener('install', event => {
3      event.waitUntil(
4          caches.open('MyCacheFile')
5              .then(cache=>{
6                  return cache.addAll(cacheurl);
7              })
8      );
9  }
10 }
11
12 self.addEventListener('fetch', event=>{
13     event.respondWith(caches.match(event.request))
14     .then(res => {
15         if (res){
16             return res;
17         }
18         return fetch(event.res);
19     })
20 })
21
```



Application

Manifest

Service workers

Storage

Storage

Local storage

Session storage

IndexedDB

Web SQL

Cookies

Private state to

Interest groups

Shared storage

Cache storage

Background services

Back/forward c

Background fet

Background syn

Bounce trackin

Notifications

Service workers

☐ Offline

☐ Update on reload

☐ Bypass for network

http://127.0.0.1:5500/

[Network requests](#)

[Update](#)

[Unregister](#)

Source

sw1.js

Received

3/26/2024, 2:23:54 PM

Status

#31 activated and is running

stop

Push

Test push message from DevTools.

Push

Sync

test-tag-from-devtools

Sync

Periodic Sync

test-tag-from-devtools

Periodic Sync

Update Cycle

Version	Update Activity	Timeline
#31	Install	
#31	Wait	
#31	Activate	

Console

What's new

top

Filter

Default levels

9 issues: 6 3 1 hidden

Service Worker registered successfully! http://127.0.0.1:5500/ app.js:4

6 Third-party cookie will be blocked. Learn more in the Issues tab.

Service workersStorage

Storage

- Local storage
- Session storage
- IndexedDB
- Web SQL
- Cookies
- Private state tokens
- Interest groups
- Shared storage
- Cache storage
 - MyCacheFile - http://127.0.0.1:5500/

Background services

- Back/forward cache
- Background fetch
- Background sync
- Bounce tracking mitigations
- Notifications
- Payment handler
- Periodic background sync
- Speculative loads
- Push messaging

Filter by Path

http://127.0.0.1:5500

Origin http://127.0.0.1:5500

Bucket name default

Is persistent No

Durability relaxed

Quota 0 B

Expiration None

#	Name	Res...	Con...	Con...	Tim...	Vary...
0	/assets/css/style.css	basic	text...	31,1...	3/2...	Origin
1	/assets/js/main.js	basic	appl...	6,364	3/2...	Origin
2	/icon/val.png	basic	ima...	4,665	3/2...	Origin
3	/index.html	basic	text...	43,3...	3/2...	Origin

Total entries: 4

ConsoleWhat's new

topFilterDefault levels9 Issues: 6 31 hidden

third-party cookie will be blocked. Learn more in the Issues tab.

Application

Manifest

Service workers

Storage

Storage

Local storage

Session storage

IndexedDB

Web SQL

Cookies

Private state token

Interest groups

Shared storage

Cache storage

MyCacheFile - 1

Background services

Back/forward cache

Background fetch

Background sync

Service workers

☒ Offline ☐ Update on reload ☐ Bypass for network

http://127.0.0.1:5500/

[Network requests](#) [Update](#) [Unregister](#)

Source [sw1.js](#) 261

Received 3/26/2024, 2:23:54 PM

Status ● #31 activated and is running [stop](#)

● #139 waiting to activate [skipWaiting](#)

Received 3/26/2024, 2:56:00 PM

Push [Push](#)

Sync [Sync](#)

Periodic Sync [Periodic Sync](#)

Update Cycle

Version	Update Activity	Timeline
#31	Install	
#31	Wait	

Console

What's new

top

Filter

Default levels

9 Issues: 6 3 1 hidden

Service Worker registered successfully! [http://127.0.0.1:5500/](#) [app.js:4](#)

6

Third-party cookie will be blocked. Learn more in the Issues tab.

Service workersStorage

Storage

- Local storage
- Session storage
 - IndexedDB
 - Web SQL
- Cookies
 - Private state tokens
 - Interest groups
- Shared storage
- Cache storage
 - MyCacheFile - http://127.0.0.1:5500/

Background services

- Back/forward cache
- Background fetch
- Background sync

Filter by Path

http://127.0.0.1:5500

Origin http://127.0.0.1:5500

Bucket name default

Is persistent No

Durability relaxed

Quota 0 B

Expiration None

#	Name	Res...	Con...	Con...	Tim...	Vary...
0	/index.html	basic	text...	43,3...	3/2...	Origin

Total entries: 1

Console: What's new

top

Filter

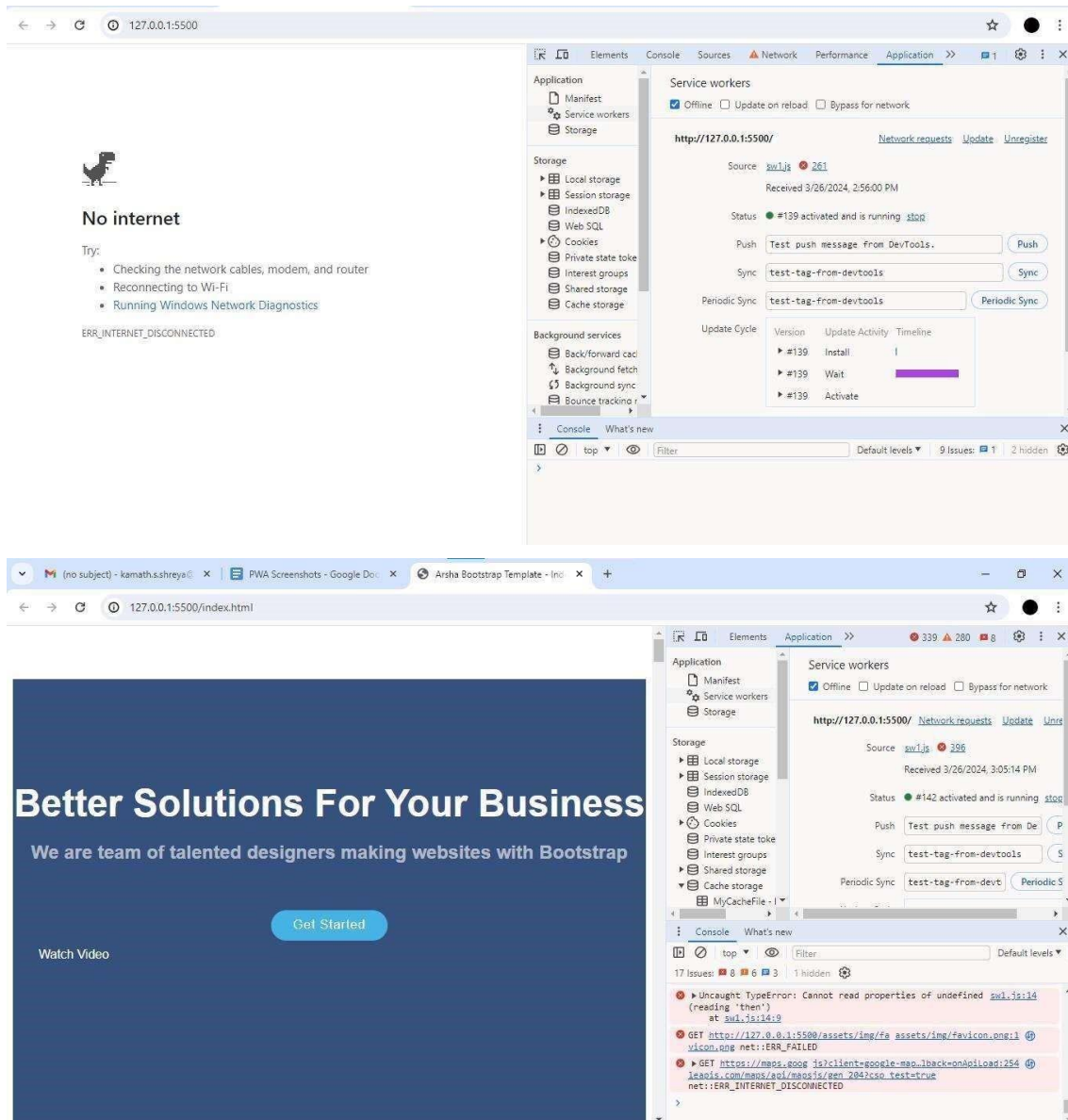
Default levels

9 Issues: 6 3

1 hidden

Service Worker registered successfully! <http://127.0.0.1:5500/> [app.js:4](#)

6 Third-party cookie will be blocked. Learn more in the Issues tab.



Conclusion: This assignment provided hands-on experience in enhancing a Progressive Web App (PWA) by implementing a service worker for caching and fetching web resources. By configuring the service worker to cache essential assets and manage fetch requests efficiently, I improved the app's performance, reliability, and offline capabilities. This exercise deepened my understanding of how service workers function behind the scenes and their critical role in delivering a seamless user experience, even in low or no network conditions.

LO Mapping: LO6